

Esercizio 2: Decompressione

Riprendendo il progetto che hai creato per esercizio file compressione, scrivi un secondo script che legge `compresso.txt` nella cartella `dati` e ricostruisce il testo originale in `decompresso.txt`.

Crea il file `decompressore.py` dentro la cartella `codice/`.

La struttura del progetto è la stessa dell'esercizio precedente:

```
Progetto_strCompDecomp/ ├── codice/ | ├── compressore.py ← già scritto | └──  
decompressore.py ← da scrivere ora └── dati/ ├── sequenza.txt ├──  
compresso.txt ← input di questo script └── decompresso.txt ← output di  
questo script
```

Cosa deve fare lo script

1. Leggere `compresso.txt` riga per riga
2. Decomprimere ogni riga applicando la logica inversa all'RLE
3. Scrivere il risultato in `decompresso.txt`
4. Verificare che `decompresso.txt` sia identico a `sequenza.txt`

Suggerimento: come funziona la decompressione

Il formato compresso è una sequenza di coppie **numero + carattere**:

```
4a4b2C → aaaabbbbCC  
20X → XXXXXXXXXXXXXXXXXXXX  
11s10t → sssssssssstttttttttt
```

Scorri la stringa con un indice `i`. Ad ogni passo:

- raccogli tutte le **cifre** consecutive → formeranno il numero
- leggi il **carattere** che segue → ripetilo tante volte quanto dice il numero

Il numero può avere più di una cifra (es. `20`, `100`): non fermarti alla prima cifra, continua a leggere finché trovi ancora un numero.

Struttura di partenza

```
import os

radice = os.getcwd()
file_compresso = os.path.join(radice, "dati", "compresso.txt")
file_decomp = os.path.join(radice, "dati", "decompresso.txt")
file_originale = os.path.join(radice, "dati", "sequenza.txt")

def decomprimi(testo):
    # scrivi qui il tuo codice
    pass

# Lettura, decompressione, scrittura
# ...

# Verifica finale
# Confronta decompresso.txt con sequenza.txt e stampa se sono identici
```